
Rapport Projet personnel : Home Lab

Recyclage d'une station Lenovo bloquée et déploiement Debian Headless

Hicham

Mars-Avril 2026

Résumé

Ce rapport documente le processus complet de réhabilitation d'un ordinateur Lenovo Thinkcentre d'entreprise, initialement inutilisable en raison de restrictions administratives strictes (BIOS verrouillé et session Windows 7 Pro protégée). Le rapport retrace les différentes étapes de contournement matériel, l'installation par transplantation de système de fichiers et la configuration d'un environnement serveur robuste et accessible à distance.

Table des matières

1	Introduction	3
2	Exploration et Échecs de Contournement Logiciel	3
2.1	La tentative de piratage de Windows 7 Pro	3
2.2	Le mur du BIOS Supervisor	3
3	Problématiques de création de support d'installation	4
3.1	L'impasse Rufus et ISO	4
4	La Méthode de la "Transplantation" (Solution)	4
4.1	Extraction et Installation externe	4
4.2	Le "Choc" de la réinsertion	6
5	Configuration Réseau et Post-Installation	6
5.1	Le défi des Firmwares Propriétaires	6
5.2	Adressage IP Statique et Accès Headless	7
6	Déploiement des Services via Docker	7
6.1	Installation de l'environnement Docker	7
6.2	Déploiement de Jellyfin	7
7	Analyse des Obstacles Réseau et Sécurité	8
7.1	Le problème de l'isolation AP (Access Point)	8
7.2	Sécurisation locale via UFW	8
7.3	Solution envisagée : Cloudflare Tunnel	8
8	Résolution des conflits d'adressage et VPN Mesh	8
8.1	Diagnostic de l'échec de connectivité	9
8.2	Purge de la configuration statique et passage en DHCP	9
8.3	Validation du tunnel Tailscale	9
9	Architecture de la Solution : La Stack "Arr"	9
9.1	Composants Applicatifs	9
9.2	Modèle de Communication Inter-Conteneurs	10
10	Sécurisation du Réseau et Contournement du Pare-feu Campus	10
10.1	Phase d'Investigation et Choix Protocolaires	10
10.2	Résolution des Anomalies d'Authentification	10
10.3	Stabilisation et Optimisation DNS	11
11	Limites Rencontrées : Le Cas du Protocole P2P	11
11.1	Diagnostic Technique	11
12	Limites Rencontrées et Diagnostic Avancé : Le Cas du Protocole P2P	11
12.1	Isolation de la Panne : Le Test du Réseau Non Filtré	11
13	Conclusion Générale	11

1 Introduction

L'objectif de ce projet est de transformer une machine obsolète et bridée en un serveur de services auto-hébergés (Plex, Docker, Cloud Privé). La contrainte majeure résidait l'incapacité d'utiliser les méthodes de démarrage traditionnelles (USB Boot), l'impossibilité d'accéder au système d'exploitation d'origine ainsi que l'accessibilité du serveur derrière un wifi de campus bridé par un pare feu.

2 Exploration et Échecs de Contournement Logiciel

2.1 La tentative de piratage de Windows 7 Pro

La première approche consistait à tenter de récupérer l'accès à la session administrateur Windows 7 présente sur le disque.

- **Méthode** : Utilisation de la faille des "Sticky Keys" (`sethc.exe`). L'idée était de remplacer l'exécutable des touches rémanentes par une invite de commande (`cmd.exe`) via l'environnement de récupération système.
- **Procédure** : Forcer un arrêt brutal pour déclencher la réparation automatique, accéder à l'explorateur de fichiers pour renommer les fichiers système.
- **Résultat : Échec critique**. Le système était protégé par des permissions NTFS restrictives (TrustedInstaller) ou une stratégie de groupe empêchant la modification du répertoire `System32`. La tentative de bypass n'a pas permis d'obtenir l'invite de commande au démarrage.

Comme je savais que j'allais changer l'OS je n'ai pas cherché plus loin pour accéder à la session et à son contenu.

2.2 Le mur du BIOS Supervisor

Pour installer un nouvel OS, la priorité était de booter sur une clé USB. Cependant, le BIOS était protégé par un mot de passe superviseur.

- **Analyse** : L'ordre de boot était fixé dans le bios. Dans le Boot Menu l'USB n'était pas visible (surement USB boot bloqué), le bios était protégé par mot de passe empêchant toute modification.
- **Exploration physique** : Le retrait de la pile CMOS a été effectué pour tenter de vider la mémoire volatile pour réinitialiser le mot de passe du BIOS. Pour vider la mémoire il fallait allumer l'ordinateur avec la pile retirée et avoir modifié l'emplacement du cavalier (embout noir).
- **Observation technique** : Sur cette gamme Lenovo, contrairement aux PC grand public, le mot de passe est stocké dans une puce **EEPROM** non-volatile. Le retrait de la pile n'a aucun effet sur le mot de passe, confirmant l'impossibilité de modifier les réglages de boot.

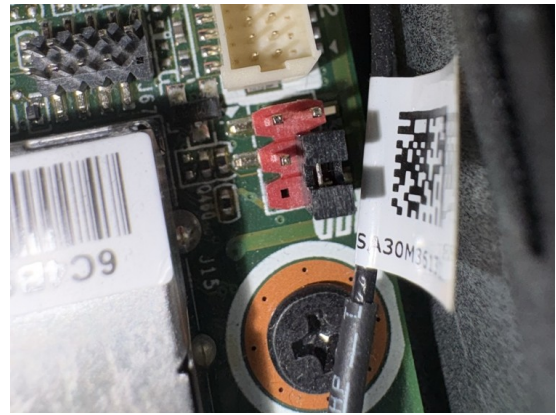
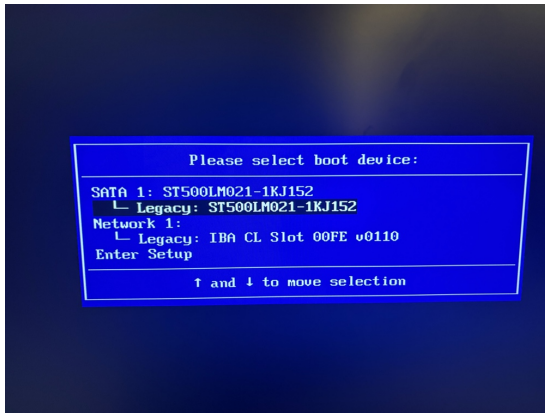


FIGURE 1 – Boot Menu et pins permettant de vidé la mémoire volatile

3 Problématiques de création de support d'installation

3.1 L'impasse Rufus et ISO

Plusieurs clés USB de démarrage ont été préparées via le logiciel **Rufus** avec différentes configurations :

- **Schéma de partition MBR vs GPT** : Test d'une configuration MBR pour la compatibilité avec le BIOS "Legacy" du Lenovo.
- **Système de destination** : BIOS (ou UEFI-CSM).
- **Résultat** : Malgré une préparation parfaite, le matériel Lenovo ignorait superbement la clé USB au démarrage.

4 La Méthode de la "Transplantation" (Solution)

La solution a consisté à externaliser le problème. Puisque je ne pouvais pas amener l'installateur au disque, j'ai amené le disque à l'installateur. En effet il est plus simple de se battre à domicile (sur mon ordinateur personnel où j'ai tous les droits).

4.1 Extraction et Installation externe

1. **Démontage** : Extraction du disque dur SATA 2.5" du Lenovo.
2. **Interface** : Connexion du disque sur mon ordinateur personnel (en adaptateur SATA vers USB).

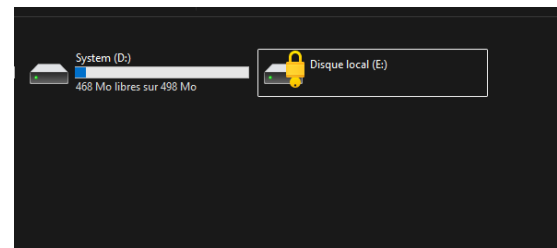


FIGURE 2 – Ordinateur sans ventirad ni disque dur et capture d'écran du disque connecté sur mon ordinateur personnel

3. **Installation de Debian :** Lancement d'une installation standard de Debian 13 sur ce disque. De nombreux essais d'installation ont été effectués, car de nombreuses erreurs d'installations sont arrivés par exemple lors de la création de partition ou le téléchargement des ressources d'installations. Des tentatives de partitionnement à la main du disque ont été effectués.

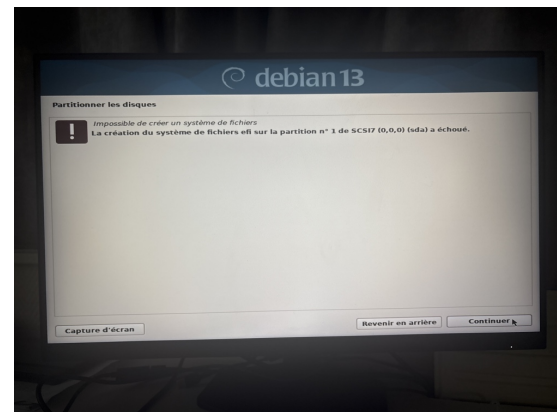
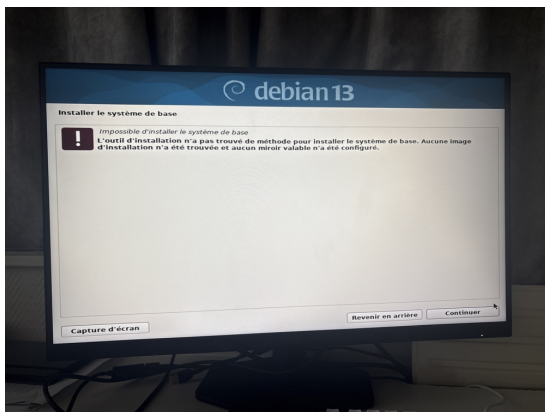


FIGURE 3 – Exemples de problèmes d'installations

4. **Solution :** La solution trouvée a été de faire tourner le boot depuis une clé USB, d'utiliser l'ISO DVD (ne nécessitant pas internet donc) et de générer la clé bootable en mode DD et non ISO. J'ai effacer la table de partitionnement à la main du disque avec cette commande.

```
dd if=/dev/zero of=/dev/sda bs=1M count=10
```

Puis j'ai suivi l'installation guidée pour avoir un partitionnement correct rapidement.

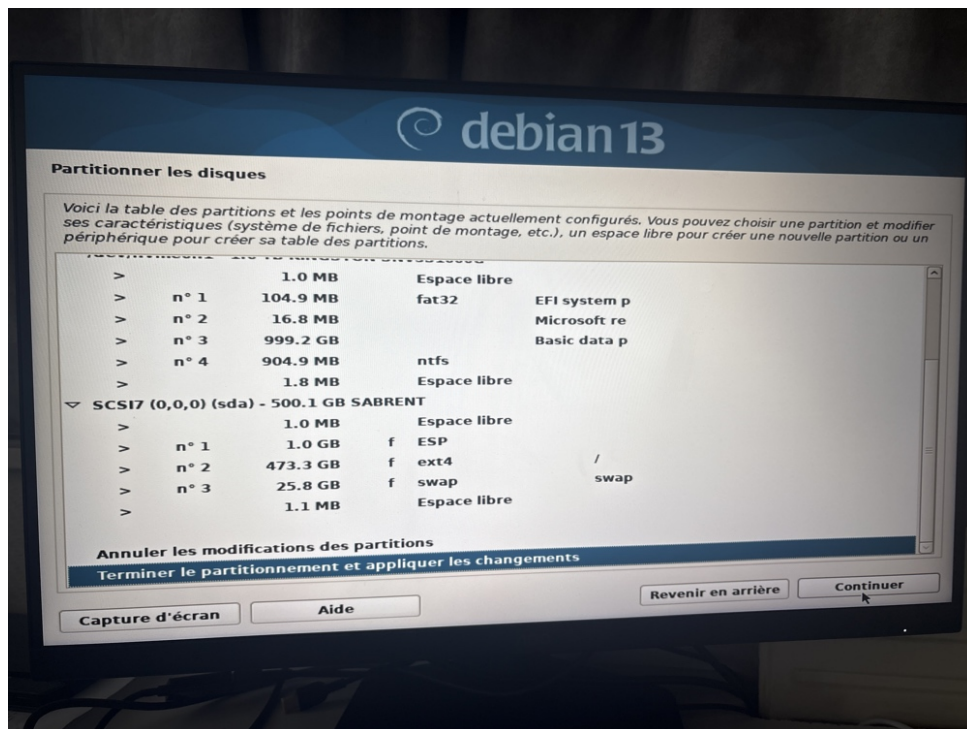


FIGURE 4 – Partitionnement du disque

5. **Configuration** : Lors de l'installation, le choix a été fait de ne pas installer d'interface graphique (mode *headless*) pour maximiser les ressources disponibles.

4.2 Le "Choc" de la réinsertion

Après réinsertion du disque dans le Lenovo :

- **Succès** : Le BIOS, programmé pour démarrer sur le disque interne, a trouvé le nouveau secteur d'amorçage GRUB. Debian a démarré avec succès.
- **Théorie** : Linux possède un noyau monolithique capable de charger les pilotes nécessaires dynamiquement lors du boot, ce qui permet à un système installé sur une machine A de démarrer sur une machine B (contrairement à Windows qui génère souvent des écrans bleus dans cette situation).

5 Configuration Réseau et Post-Installation

5.1 Le défi des Firmwares Propriétaires

Debian étant une distribution prônant le logiciel libre, les pilotes Wi-Fi propriétaires ne sont pas inclus par défaut.

- **Problème** : La carte réseau `wlp1s0` était détectée mais inutilisable ("Hardware disabled").
- **Solution** : Modification du dépôt pour inclure `non-free-firmware` et installation de `firmware-iwlwifi`.

— **Commandes :**

```
sudo apt update && sudo apt install firmware-iwlwifi  
sudo rfkill unblock all
```

5.2 Adressage IP Statique et Accès Headless

Pour garantir la pérennité de l'accès SSH, l'adresse IP a été fixée manuellement via l'outil `nmcli` (Network Manager CLI) :

- **Passerelle (Gateway)** : 10.196.14.1 (le routeur).
- **DNS** : 8.8.8.8 (Google DNS) pour assurer la résolution de noms.
- **Configuration** : Passage de la méthode DHCP à Manual.

6 Déploiement des Services via Docker

Une fois le système de base stabilisé, l'objectif était de transformer le serveur en centre multimédia personnel. Le choix s'est porté sur l'outil de conteneurisation **Docker** pour sa flexibilité et l'isolation des processus.

6.1 Installation de l'environnement Docker

L'installation a été réalisée via le script officiel de Docker. Pour simplifier la gestion des services, **Docker Compose** a été utilisé afin de définir l'infrastructure sous forme de fichier YAML.

6.2 Déploiement de Jellyfin

Jellyfin a été choisi comme alternative libre à Plex, ne nécessitant aucun compte tiers pour fonctionner.

- **Configuration** : Utilisation du `network_mode: host` pour simplifier la découverte DLNA sur le réseau local.
- **Volume** : Liaison (*bind mount*) entre le répertoire de médias du disque et le conteneur.

```
services:  
  jellyfin:  
    image: jellyfin/jellyfin:latest  
    container_name: jellyfin  
    network_mode: host  
    restart: unless-stopped  
    volumes:  
      - /home/hicham/media:/data
```

Listing 1 – Extrait du fichier `docker-compose.yml`

7 Analyse des Obstacles Réseau et Sécurité

Le déploiement en environnement étudiant (PlanetCampus) a révélé des défis majeurs en matière de connectivité, nécessitant une démarche de diagnostic rigoureuse.

7.1 Le problème de l'isolation AP (Access Point)

Malgré un service Docker "Healthy", l'accès à l'interface de Jellyfin via `http://10.196.14.177:8096` depuis une machine tierce a échoué systématiquement (*Connection Timed Out*).

- **Diagnostic** : Utilisation d'un partage de connexion mobile (iPhone) pour créer un réseau local privé (LAN) sans restrictions.
- **Observation** : Le succès du `ping` entre les deux machines sur ce nouveau réseau a confirmé que le routeur du campus pratiquait l'**isolation de clients**, empêchant toute communication directe entre appareils.

7.2 Sécurisation locale via UFW

Un second obstacle a été identifié au niveau de la couche logicielle du serveur : le pare-feu **UFW** bloquait les flux entrants sur le port applicatif.

- **Action** : Ouverture du port 8096.
- **Vérification** : Utilisation de la commande `ss -tulpn` pour confirmer que le service écoutait bien sur toutes les interfaces (`0.0.0.0`).

```
sudo ufw allow 8096/tcp
sudo ufw reload
```

7.3 Solution envisagée : Cloudflare Tunnel

Pour s'affranchir définitivement des contraintes de NAT et d'isolation du campus, la mise en place d'un **Cloudflare Tunnel** a été initiée.

- **Concept** : Le serveur établit une connexion sortante persistante vers les serveurs de Cloudflare, agissant comme un pont inverse (*Reverse Proxy*).
- **Avantage** : Permet l'accès au serveur via un nom de domaine (ex : *jellyfin.mondomaine.fr*) sans ouverture de port entrant sur le routeur du campus, tout en bénéficiant d'un chiffrement TLS (HTTPS).

Cette méthode a été abandonnée car je ne possède pas de nom de domaine.

8 Résolution des conflits d'adressage et VPN Mesh

L'accès distant via **Tailscale** a nécessité une correction majeure de la couche réseau physique pour fonctionner derrière l'infrastructure *PlanetCampus*.

8.1 Diagnostic de l'échec de connectivité

Après avoir configuré une IP statique, le serveur est devenu **offline**. L'analyse par **ping** a révélé une erreur *"Destination Host Unreachable"* émanant de l'interface locale. Ce symptôme indiquait que le serveur tentait de communiquer via une adresse IP (10.196.14.177) non reconnue ou bloquée par le routeur du campus.

8.2 Purge de la configuration statique et passage en DHCP

Pour rétablir l'accès à Internet et permettre au tunnel VPN de se monter, une réinitialisation complète de l'interface **wlp1s0** a été effectuée via **nmcli** :

```
# Suppression de l'adresse et de la passerelle statiques
sudo nmcli connection modify "PlanetCampus-Prive016752" ipv4.
    addresses "" ipv4.gateway ""

# Activation du mode d'adressage dynamique (DHCP)
sudo nmcli connection modify "PlanetCampus-Prive016752" ipv4.
    method auto

# Application de la nouvelle configuration
sudo nmcli connection up "PlanetCampus-Prive016752"
```

8.3 Validation du tunnel Tailscale

Une fois l'accès Internet rétabli (validé par un **ping** vers 8.8.8.8), le service Tailscale a pu synchroniser son état avec les serveurs de coordination.

- **Adressage persistant** : Le serveur dispose désormais d'une IP fixe virtuelle 100.127.159.128.
- **Accessibilité** : Cette adresse permet une connexion SSH stable depuis n'importe quel réseau, s'affranchissant des changements d'IP locale imposés par le DHCP du campus.
- **Services** : L'interface Web de Jellyfin est désormais accessible via le port 8096 sur l'IP du tunnel.

Je peux me permettre de mettre l'adresse IP sur le rapport car même si vous l'avez vous ne pourrez pas vous y connecter sans avoir accès au compte Tailscale qui fait le pont.

9 Architecture de la Solution : La Stack "Arr"

Pour répondre au besoin d'automatisation, j'ai utilisé l'écosystème dit "Arr", une suite de micro-services spécialisés communiquant entre eux.

9.1 Composants Applicatifs

- **Jellyfin** : Le serveur de média frontal (Frontend) assurant l'indexation et le streaming.
- **qBittorrent** : Le client de transport pour le protocole BitTorrent.

- **Radarr & Sonarr** : Gestionnaires de recherche automatisée pour les films et les séries.
- **Prowlarr** : Indexeur centralisant les flux de recherche pour Radarr/Sonarr.
- **Jellyseerr** : Interface de requête simplifiée pour l'utilisateur final.

9.2 Modèle de Communication Inter-Conteneurs

L'une des difficultés techniques a été la gestion du réseau interne Docker. En utilisant le mode `network_mode: "service:gluetun"`, tous les services de téléchargement partagent la même interface réseau virtuelle.

Choix technique : Pour assurer la communication entre Radarr et qBittorrent au sein de ce tunnel, l'utilisation de l'adresse de boucle locale (127.0.0.1) a été privilégiée par rapport aux noms de services Docker classiques, car ils partagent désormais le même espace de nommage réseau (Network Namespace).

10 Sécurisation du Réseau et Contournement du Pare-feu Campus

La partie la plus critique du projet a concerné le déploiement du client VPN (Gluetun), agissant comme une passerelle obligatoire pour tout le trafic P2P.

10.1 Phase d'Investigation et Choix Protocolaires

Le réseau étudiant PlanetCampus bloque par défaut les protocoles UDP non-standards et le chiffrement DNS (DoT sur port 853). En effet, les tentatives initiales en OpenVPN UDP et WireGuard ont échoué (i/o timeout).

Solution : J'ai opté pour une encapsulation **OpenVPN sur protocole TCP via le port 443**. *Justification* : Le trafic VPN est ainsi masqué sous l'apparence d'un flux HTTPS standard, rendant son blocage impossible pour le pare-feu sans interrompre l'accès web général.

10.2 Résolution des Anomalies d'Authentification

L'accès au service ProtonVPN a généré des erreurs `AUTH_FAILED` persistantes. L'analyse des logs et de la documentation a permis d'identifier la cause : les identifiants Proton Free ne fonctionnent que sur des serveurs spécifiques. L'ajout de `FREE_ONLY=on` en option dans le docker compose a été nécessaire.

10.3 Stabilisation et Optimisation DNS

Pour éviter les redémarrages cycliques du conteneur Gluetun dus aux échecs de test de santé (*healthchecks*), j'ai fait les correctifs suivants :

```
environment:
- HEALTH_VPN_SUCCESS_THRESHOLD=0
- DOT=off # Desactivation du DNS chiffré (bloque par le campus)
- DNS_KEEP_NAMESERVER=on # Utilisation du DNS interne au tunnel
- VPN_INTERFACE_SKIP_CHECK=on # Eviter les conflits de tables de
  routage
```

Listing 2 – Extrait de configuration Gluetun

11 Limites Rencontrées : Le Cas du Protocole P2P

Après avoir stabilisé le tunnel VPN (confirmé par l'obtention d'une adresse IP localisée aux Pays-Bas dans les logs), un blocage au niveau de la couche transport est apparu : les téléchargements restaient à 0%.

11.1 Diagnostic Technique

Malgré une connexion VPN active (obtention d'une IP publique localisée à l'étranger) j'avais une absence de nœuds (*peers*) dans qBittorrent. **Hypothèse** : ProtonVPN, dans sa version gratuite, pratique un filtrage de paquets (Deep Packet Inspection) pour bloquer les flux BitTorrent sur ses serveurs gratuits. Le tunnel est fonctionnel pour la navigation web (TCP 443), mais le protocole P2P est activement rejeté.

12 Limites Rencontrées et Diagnostic Avancé : Le Cas du Protocole P2P

12.1 Isolation de la Panne : Le Test du Réseau Non Filtré

Face aux erreurs persistantes telles que `Operation not permitted` et `Requested download is not authorized` et à l'absence de nœuds. J'ai mise en place une stratégie pour déterminer l'origine exacte du blocage (encore une fois le pare-feu du campus vs Fournisseur VPN) : un test d'isolement a été réalisé en basculant la machine hôte sur un réseau mobile cellulaire (Partage de connexion 4G), dépourvu de tout filtrage. **Conclusion du test** : L'impossibilité de télécharger, même sur un réseau ouvert, confirme de manière irréfutable que le blocage est externe. Le fournisseur VPN (dans ses offres gratuites) applique des Access Control Lists (ACL) strictes qui rejettent activement les paquets UDP relatifs au trafic P2P sûrement afin de préserver la bande passante de ses serveurs.

13 Conclusion Générale

Ce projet démontre qu'avec une compréhension approfondie des couches matérielles et logicielles, il est possible de recycler du matériel verrouillé. L'expérience a permis de

confronter les concepts théoriques de réseau (modèle OSI, NAT, Firewall) à la réalité technique d'un environnement hostile (réseau de campus). Bien que le cas d'usage final (le téléchargement automatisé via P2P) se heurte aux politiques commerciales des fournisseurs VPN gratuits, ce projet a constitué un cas d'étude en ingénierie système et réseaux et une application de la théorie vu en cours. L'infrastructure déployée est techniquement viable et les objectifs pédagogiques ont été pleinement atteints :

- **Maîtrise de la conteneurisation** : Déploiement automatisé d'une stack micro-services complexe (Jellyfin, *arr, Torrent) via Docker Compose.
- **Utilisation de notion réseau** : Manipulation des Network Namespaces, résolution de conflits de ports, et contournement des restrictions d'un réseau restrictif par obfuscation de trafic (OpenVPN encapsulé en TCP 443).
- **Sécurité et Résilience** : Implémentation d'un environnement étanche où la perte du VPN bloque instantanément la connectivité des services dépendants, empêchant toute compromission de l'adresse IP réelle (Kill Switch).

Pour le passage en production, l'architecture est finalisée. L'activation du service nécessitera simplement l'adoption de l'une de ces deux solutions :

1. La souscription à un fournisseur VPN Premium autorisant explicitement le P2P et la redirection de port (*Port Forwarding*), tel que Mullvad, ProtonVPN Plus ou AirVPN.
2. La transition vers un modèle de type "Seedbox" hébergée, rapatriant les médias de manière sécurisée (SFTP/HTTPS), s'affranchissant ainsi totalement de l'usage du P2P sur le réseau du campus.

Ce projet m'a permis de mieux comprendre la gestion de périphérique et des disques ainsi que leur fonctionnement sous Debian, mais aussi d'approfondir avec de la pratique mes connaissances en réseaux.